Git dan GitHub untuk Pemula

Panduan langkah demi langkah untuk mahasiswa yang belum pernah memakai Git



Contoh tampilan yang akan Anda lihat sepanjang panduan ini.

Cara memakai panduan ini. Setiap langkah berisi (1) perintah yang harus diketik, (2) penjelasan singkat, dan (3) contoh tampilan hasilnya. Jangan hanya membaca — ketik sendiri perintahnya di komputer Anda secara berurutan. Perintah yang diawali tanda # hanyalah keterangan dan tidak perlu diketik.

Mata kuliah	Pengembangan Aplikasi Berbasis Web (PABW) · SIF302
Dipakai mulai	Pertemuan 4 — dan dipakai terus sampai Pameran Eksposisi
Pengampu	Dr. Hendrik, S.T., M.Eng. · hendrik@uii.ac.id
Program studi	Program Studi Informatika — Universitas Islam Indonesia
Waktu belajar	± 45 menit untuk Bagian 1–3, sisanya dibaca saat dibutuhkan

Isi panduan

Bagian 1 **Memahami Git dan GitHub** — Apa masalah yang diselesaikan, dan bedanya Git dengan GitHub

Bagian 2 **Menyiapkan perkakas dan akun** — Instalasi Git, membuat akun GitHub, mengatur identitas dan akses aman

Bagian 3 **Sembilan langkah pertama** — Dari membuat repositori sampai menyelesaikan konflik — semuanya dicoba

Bagian 4 **Alur kerja kelompok** — Branch, pull request, review, dan merge tanpa saling menimpa

Bagian 5 **Mengumpulkan tugas lewat GitHub Classroom** — Menerima tugas, mengerjakan, dan memastikan unggahan berhasil

Bagian 6 **Kesalahan umum dan cara memperbaikinya** — Empat belas masalah yang paling sering terjadi beserta solusinya

Bagian 7 **Kartu ringkasan perintah** — Satu halaman untuk ditempel di dekat meja belajar

BAGIAN 1

Memahami Git dan GitHub

1.1 Masalah yang diselesaikan Git

Coba ingat bagaimana Anda menyimpan tugas selama ini. Biasanya hasilnya seperti ini:

```
proposal.docx
proposal_revisi.docx
proposal_revisi2.docx
proposal_revisi2_FINAL.docx
proposal_revisi2_FINAL_beneran.docx
proposal_revisi2_FINAL_beneran_edit_dosen.docx
proposal_revisi2_FINAL_beneran_edit_dosen(1).docx
```

Cara itu menimbulkan empat masalah yang serius, terutama kalau dikerjakan berkelompok:

- **Kebingungan versi.** Tidak ada yang tahu versi mana yang terbaru. Setiap anggota punya salinan sendiri.
- **Tidak bisa mundur.** Kalau kalimat paragraf ketiga rusak, Anda tidak bisa kembali ke versi kemarin. Salinan lama biasanya sudah tertimpa.
- **Pekerjaan hilang.** Kalau dua orang mengubah berkas yang sama, salah satu pekerjaan pasti hilang karena harus menyalin manual.
- **Tidak ada jejak.** Tidak ada catatan siapa mengubah apa, kapan, dan mengapa.

Git menyelesaikan keempat masalah itu. Git adalah pencatat perubahan yang bekerja di dalam komputer Anda. Setiap kali Anda menyimpan perubahan secara resmi — disebut commit — Git mengambil gambar (snapshot) seluruh proyek pada saat itu, memberi pesan penjelasan, dan menyimpannya sebagai riwayat. Berkasnya tetap satu saja. Tidak ada lagi proposal_revisi2_FINAL_beneran.docx.

Analogi yang berguna. Bayangkan Git sebagai mesin waktu untuk folder proyek Anda. Setiap commit adalah titik yang bisa Anda datangi kembali. Kalau percobaan hari ini gagal, Anda bisa kembali ke titik commit kemarin tanpa kehilangan apa pun.

1.2 Beda Git dan GitHub

Dua istilah ini sering tertukar, padahal keduanya berbeda. Git adalah program yang sudah terpasang di komputer Anda. GitHub adalah situs web tempat salinan proyek Anda ditaruh secara daring supaya bisa diakses dari komputer lain, dibagikan ke dosen, dan dikerjakan bersama anggota kelompok.

	Git	GitHub
Bentuknya	Program yang dijalankan di terminal komputer Anda	Layanan web (situs) milik Microsoft
Perlu internet?	Tidak. Semua riwayat tersimpan di komputer Anda sendiri	Ya. Dipakai untuk berbagi dan mencadangkan
Fungsi utama	Mencatat riwayat perubahan berkas	Menyimpan salinan daring, mengatur kerja sama, menampilkan riwayat
Perintahnya	git add, git commit, git log, git branch	Tombol dan menu di situs github.com (disebut juga operasi server)
Kalau tidak ada	Tidak ada riwayat perubahan lokal	Proyek tetap jalan sendiri, tetapi tidak bisa dibagikan
Padanan mudah	Mesin waktu di komputer Anda	Lemari arsip di internet

Kesimpulan penting. Git tidak butuh GitHub, tetapi GitHub tidak berguna tanpa Git. Yang menyimpan

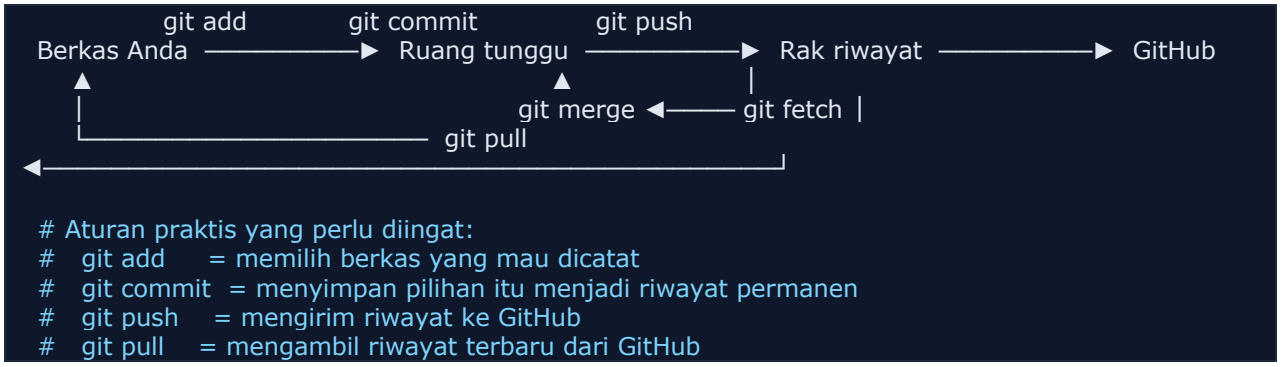
riwayat adalah Git; GitHub hanya tempat menaruh salinannya. Itu sebabnya langkah pertama selalu memasang Git, bukan membuat akun GitHub.

1.3 Tiga tempat penyimpanan di Git

Setiap kali Anda menyimpan pekerjaan, berkas Anda menempuh tiga tempat. Memahami tiga tempat ini adalah kunci untuk tidak tersesat, karena sebagian besar kebingungan pemula berasal dari tidak tahu berkasnya sekarang berada di mana.

Tempat	Nama aslinya	Isinya	Perintah untuk memindahkan
Tempat kerja	Working directory	Berkas yang sedang Anda edit. Belum dicatat Git.	Mengedit langsung dengan editor teks
Ruang tunggu	Staging area	Perubahan yang sudah dipilih untuk masuk ke commit berikutnya. Bisa diperiksa dulu.	git add nama-berkas
Rak riwayat	Local repository	Commit yang sudah permanen dan tersimpan di riwayat komputer Anda.	git commit -m "pesan"
Situs GitHub	Remote repository	Salinan di server GitHub, agar bisa diakses orang lain dan komputer lain.	git push

Alurnya digambarkan seperti ini. Dari kiri ke kanan berarti mengirim pekerjaan keluar menuju GitHub; dari kanan ke kiri berarti mengambil pekerjaan orang lain masuk ke komputer Anda.



1.4 Istilah yang akan terus muncul

Delapan istilah berikut dipakai sepanjang panduan ini. Tidak perlu dihafal sekarang; baca sekali, lalu kembali ke sini kalau ada yang lupa. Daftar lengkapnya ada di Lampiran B.

Istilah	Arti sederhana
Repositori (repo)	Satu folder proyek yang di dalamnya Git sudah dipasang. Berisi berkas proyek dan seluruh riwayatnya.
Commit	Satu titik dalam riwayat: gambar seluruh proyek pada satu waktu, dengan pesan penjelasan. Punya kode unik seperti a3f91c2.
Branch (cabang)	Jalur kerja terpisah. Anda bisa mencoba sesuatu tanpa mengganggu hasil kerja utama di branch main.
main	Nama branch utama. Di sinilah versi resmi proyek berada.
Push / pull	Push = mengirim commit Anda ke GitHub. Pull = mengambil commit terbaru dari GitHub.
Clone	Menyalin seluruh repositori beserta riwayatnya dari GitHub ke komputer baru, sekali saja di awal.
Remote	Julukan untuk alamat repositori di GitHub, biasanya origin.
Pull request	Permintaan menggabungkan branch Anda ke branch main, lengkap dengan tampilan perbedaannya untuk diperiksa anggota lain.

BAGIAN 2

Menyiapkan perkakas dan akun

Bagian ini dikerjakan sekali saja di awal semester. Setelah selesai, Anda tidak perlu mengulanginya lagi.

2.1 Memasang Git

Pilih cara sesuai sistem operasi komputer Anda. Kalau tidak yakin, lihat menu Tentang (About) di komputer Anda.

Windows

1. **https://git-scm.com/download/win** — unduh penginstalnya. Berkasnya bernama Git-x.y.z-64-bit.exe.
2. **Jalankan penginstal dengan menekan Next terus.** — Semua pilihan bawaan sudah benar. Jangan ubah apa pun.
3. **Biarkan pilihan "Git Bash Here" tetap tercentang.** — Git Bash adalah terminal yang akan kita pakai. Ini paling penting.
4. **Selesai, lalu buka Git Bash dari menu Start.** — Cari "Git Bash" di menu Start, klik sekali, sebuah jendela hitam akan terbuka.

Kenapa Git Bash, bukan Command Prompt? Git Bash memberi Anda perintah yang sama persis dengan Linux dan macOS. Perintah seperti cd, ls, dan mkdir bekerja di dalamnya. Kalau Anda memakai Command Prompt biasa, perintahnya berbeda dan mudah membingungkan. Gunakan Git Bash agar panduan ini bisa diikuti apa adanya.

macOS

1. **brew install git** — bila Homebrew sudah terpasang. Ini cara paling rapi.
2. **Atau jalankan git --version di Terminal.** — macOS akan menawarkan memasang Command Line Developer Tools. Klik Install dan tunggu selesai.

Linux (Ubuntu, Linux Mint, Debian)

1. **sudo apt update && sudo apt install git** — untuk keluarga Debian dan Ubuntu.
2. **sudo dnf install git** — untuk Fedora. Untuk Arch: sudo pacman -S git.

2.2 Memeriksa hasil pemasangan

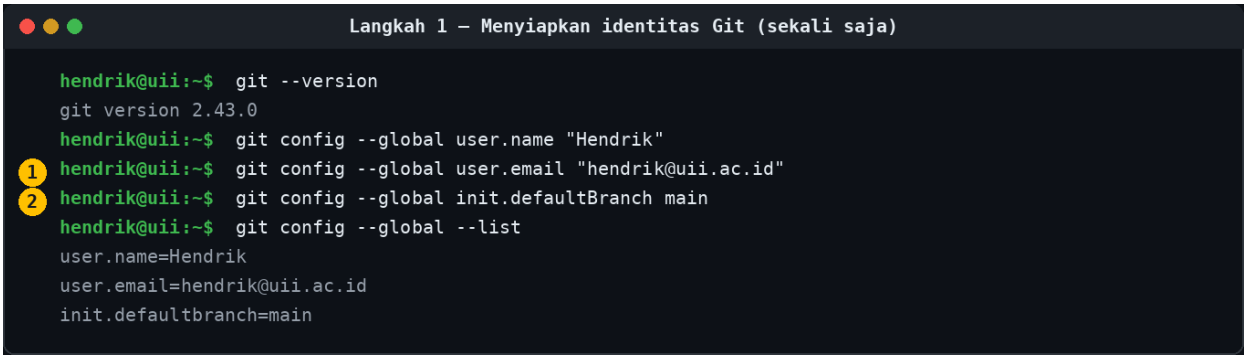
Buka terminal (Git Bash di Windows, Terminal di macOS dan Linux), lalu ketik perintah berikut. Kalau muncul nomor versi, pemasangan berhasil.

```
git --version

# Contoh keluaran yang benar:
#   git version 2.43.0
```

```
git status

# Kalau belum berada di dalam repositori, Git menjawab:
#   fatal: not a git repository (or any of the parent directories): .git
# Itu bukan kerusakan — artinya Anda belum berada di folder proyek.
```



Menjalankan git --version dan mengatur identitas Git.
Contoh keluaran nyata pada Linux/Git Bash. Tampilan Windows identik, hanya warna prompt-nya berbeda.

2.3 Mengatur identitas Anda

Ini wajib dikerjakan sebelum commit pertama. Identitas inilah yang dicatat Git pada setiap commit, sehingga dosen dan anggota kelompok tahu siapa mengerjakan apa. Bagian --global berarti pengaturan ini berlaku untuk semua proyek di komputer Anda, jadi cukup sekali seumur pemasangan.

```
git config --global user.name "Nama Anda"
git config --global user.email "email@uii.ac.id"
git config --global init.defaultBranch main

# Periksa bahwa pengaturan tersimpan:
git config --global --list
```

Pakai email yang sama dengan akun GitHub Anda. GitHub mencocokkan alamat email pada commit dengan alamat email pada akun. Kalau berbeda, kontribusi Anda tidak akan terhitung sebagai milik Anda dan tidak muncul di halaman kontribusi.

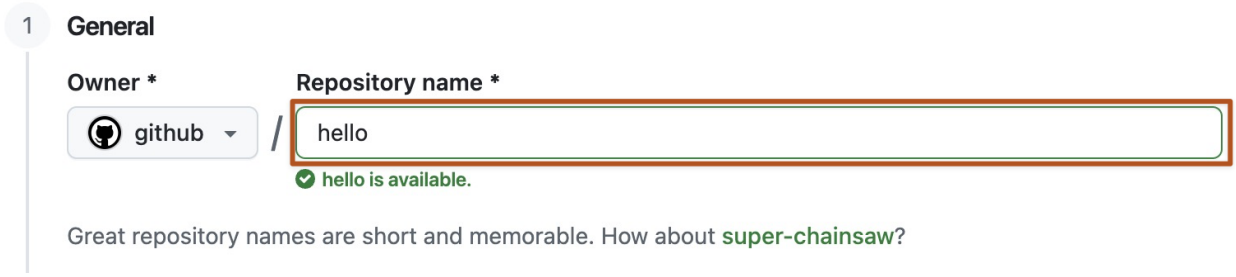
2.4 Membuat akun GitHub

Kalau sudah punya akun GitHub, lewati bagian ini. Kalau belum, ikuti langkah berikut. Akun ini akan Anda pakai sampai lulus, jadi pilih nama pengguna yang pantas dan mudah diingat — nama pengguna ini akan muncul di semua tautan repositori Anda.

1. **Buka https://github.com/signup.** — Anda akan diminta memasukkan email, kata sandi, nama pengguna, dan menjawab verifikasi.
2. **Masukkan email yang aktif.** — Gunakan email kampus atau email pribadi yang Anda periksa setiap hari. Verifikasi dikirim ke sini.
3. **Buat kata sandi yang kuat.** — Minimal delapan karakter, mengandung angka dan huruf kecil.
4. **Pilih nama pengguna yang sopan.** — Contoh: hendrik-uii, bukan nama panggilan yang tidak jelas. Nama ini muncul di URL repositori Anda.
5. **Selesaikan verifikasi gambar, lalu cek email.** — Masukkan kode yang dikirim GitHub (biasanya delapan angka).
6. **Pilih paket Free.** — Gratis dan semua fitur yang kita perlukan sudah tersedia. Jangan memilih yang berbayar.

2.5 Membuat repositori pertama di GitHub

Repositori di GitHub adalah wadah proyek Anda. Mari buat satu untuk latihan. Bagian 3 nanti akan mengisi repositori ini dengan berkas sungguhan.



Mengisi nama repositori. Status hijau "is available" berarti nama itu belum dipakai.
Sumber gambar: GitHub Docs — CC BY 4.0

Setelah menekan tombol hijau New pada halaman utama GitHub, Anda akan diminta mengisi beberapa hal. Pilih seperti berikut:

Kolom	Isi dengan	Alasan
Repository name	pabw-praktikum	Huruf kecil, pakai tanda hubung untuk spasi. Jangan pakai spasi.
Description	Latihan PABW (boleh dikosongkan)	Keterangan singkat agar Anda ingat isinya.
Public / Private	Public	Gratis dan bisa dilihat dosen tanpa diundang. Untuk tugas kelompok, Public juga memudahkan.
Add a README file	Centang	Supaya repositori tidak kosong. Repositori kosong membingungkan saat pertama kali dipakai.
.gitignore	Boleh dikosongkan dulu	Akan kita isi sendiri di Bagian 3 langkah 6, supaya Anda paham fungsinya.
License	Boleh dikosongkan	Tidak diperlukan untuk tugas kuliah.

2.6 Mengatur akses aman dari komputer Anda

Supaya bisa mengirim (push) dari komputer ke GitHub, GitHub harus mengenali Anda. Ada dua cara. Pilih salah satu saja; cara kedua lebih aman dan tidak perlu diulang setiap tahun.

	Cara 1 — Personal Access Token (paling mudah)	Cara 2 — Kunci SSH (lebih aman)
Cocok untuk	Pemula, sekali coba	Dipakai jangka panjang
Cara kerja	Anda membuat kode sandi panjang di GitHub, lalu memakainya sebagai pengganti kata sandi saat push	Komputer Anda menyimpan sepasang kunci rahasia; GitHub mengenali komputernya, bukan kata sandi
Perlu diulang?	Ya, masa berlakunya terbatas (pilih 90 hari atau lebih)	Tidak, kunci bertahan bertahun-tahun
Risiko	Token mudah bocor kalau ditempel di berkas atau dikirim ke orang lain	Aman selama berkas kunci di komputer tidak disalin ke tempat lain

Cara 1 — Personal Access Token

1. **Di GitHub, klik foto profil → Settings.** — Ada di kanan atas halaman.
2. **Gulir ke bawah, pilih Developer settings.** — Biasanya di paling bawah daftar menu kiri.
3. **Pilih Personal access tokens → Tokens (classic) → Generate new token (classic).** — Gunakan varian classic agar pilihannya sederhana.
4. **Beri nama, misalnya laptop-kuliah, dan tetapkan masa berlaku.** — Jangan pilih No expiration; tetapkan 90 hari.
5. **Centang scope repo.** — Ini memberi izin membaca dan menulis repositori.
6. **Klik Generate token, lalu SALIN tokennya segera.** — Token hanya ditampilkan sekali. Kalau hilang, harus dibuat ulang.

7. **Simpan di pengelola kata sandi.** — Jangan simpan di berkas proyek, jangan kirim lewat WhatsApp atau grup kelas.

Saat push pertama, Git akan meminta Username dan Password. Isi Username dengan nama pengguna GitHub Anda, dan Password dengan token yang tadi disalin — bukan kata sandi akun Anda.

Token itu seperti kunci rumah. Jangan pernah ditempel di berkas proyek. Kalau token terlanjur tersimpan di berkas lalu ter-push ke GitHub, siapa pun yang melihat repositori publik Anda dapat memakainya. Kalau itu terjadi, segera buka Settings → Developer settings dan klik Revoke (batalkan) token tersebut, lalu buat yang baru.

Cara 2 — Kunci SSH (disarankan untuk jangka panjang)

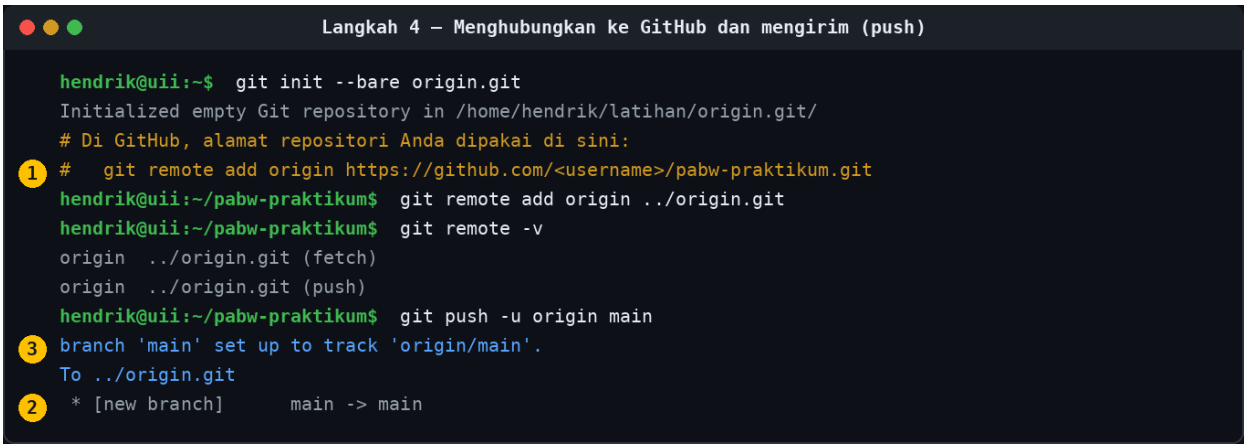
Cara ini memakai sepasang kunci: satu tersimpan di komputer Anda (rahasia), satu ditempelkan ke GitHub (publik). Setelah selesai, Anda tidak akan diminta kata sandi lagi.

```
# 1. Buat sepasang kunci. Tekan Enter terus untuk menerima lokasi bawaan.
ssh-keygen -t ed25519 -C "email@uii.ac.id"

# 2. Tampilkan kunci publik lalu salin seluruh isinya.
cat ~/.ssh/id_ed25519.pub

# 3. Tempelkan hasil salinan ke GitHub:
#    Settings -> SSH and GPG keys -> New SSH key -> Paste

# 4. Uji apakah sudah dikenali. Balasan yang benar memuat nama Anda.
ssh -T git@github.com
```



Menghubungkan repositori lokal dengan repositori di GitHub, lalu mengirim commit pertama.
Contoh keluaran nyata. Alamat ../origin.git di sini menunjuk repositori contoh di komputer yang sama; pada tugas Anda, alamatnya berupa https://github.com/<nama-pengguna>/<nama-repo>.git

BAGIAN 3

Sembilan langkah pertama

Bagian ini adalah inti panduan. Kerjakan berurutan di komputer Anda sendiri. Semua tampilan yang ditampilkan di bawah adalah keluaran sungguhan dari perintah yang dijalankan pada contoh proyek bernama pabw-praktikum, jadi apa yang Anda lihat nanti akan mirip sekali.

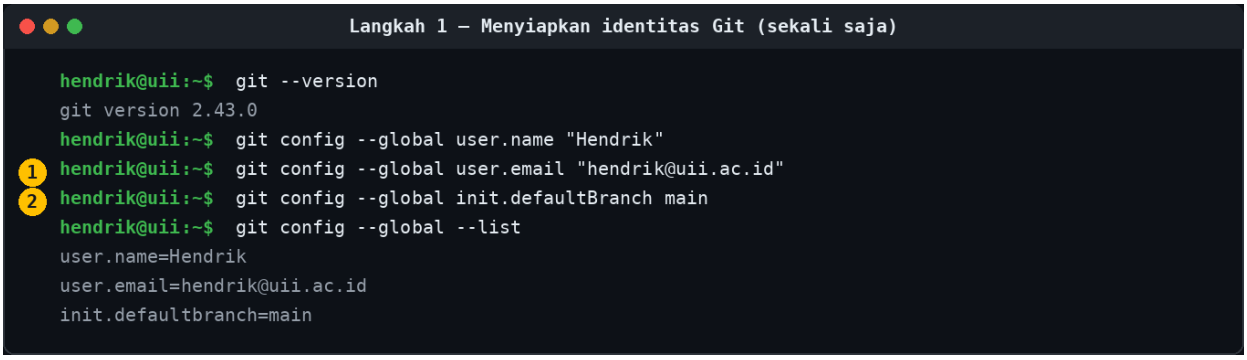
Aturan satu-satunya untuk bagian ini. Kalau ada perintah yang gagal, jangan lanjut ke langkah berikutnya. Setiap langkah mengandaikan langkah sebelumnya sudah berhasil. Lihat Bagian 6 untuk mencari pesan kegagalan yang Anda terima.

Langkah 1 — Mengatur identitas Git

Dikerjakan sekali saja seumur komputer. Lewati kalau Anda sudah melakukannya di Bagian 2.3.

```
git config --global user.name "Nama Anda"
git config --global user.email "email@uii.ac.id"
git config --global init.defaultBranch main
git config --global --list
```

Perhatikan bahwa tulisan nama pengguna Anda harus diapit tanda kutip. Tanda kutip itu memberi tahu Git bahwa semuanya, termasuk spasi, adalah satu kesatuan nama.



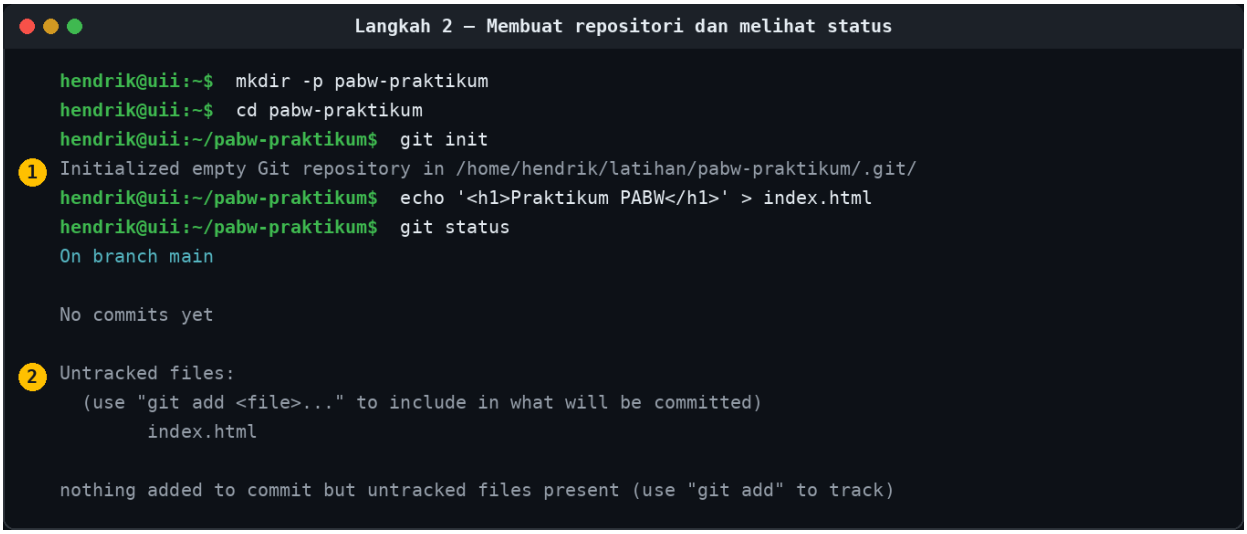
Langkah 2 — Membuat repositori dan melihat statusnya

Repositori adalah folder proyek yang sudah dipasangi Git. Perintah git init dijalankan di dalam folder proyek, sekali saja untuk selamanya. Setelah itu, folder biasa berubah menjadi folder yang dilacak Git.

```
mkdir pabw-praktikum      # buat foldernya
cd pabw-praktikum         # masuk ke dalamnya
git init                  # pasang Git di folder ini

echo "<h1>Praktikum PABW</h1>" > index.html
git status                # lihat apa yang Git amati
```

Git akan melaporkan bahwa index.html belum dilacak. Istilah Untracked files berarti berkas itu ada di komputer Anda, tetapi belum pernah dicatat dalam riwayat. Perhatikan bahwa Git sudah menyarankan perintah berikutnya, yaitu git add.



Langkah 3 — Menyimpan perubahan dengan commit

Menyimpan pada Git berlangsung dua tahap. Tahap pertama git add memasukkan berkas ke ruang tunggu; tahap kedua git commit menyimpannya sebagai riwayat permanen. Tahap pertama sengaja dipisah agar Anda bisa memeriksa dulu apa yang akan tersimpan.

```
git add index.html       # masukkan berkas ke ruang tunggu
git status               # periksa: berkas sudah siap?
git commit -m "Tambah halaman index"    # simpan sebagai riwayat
git log --oneline        # lihat riwayat yang sudah ada
```

Perhatikan keluaran git status berubah menjadi Changes to be committed. Inilah tandanya berkas sudah berada di ruang tunggu. Setelah commit, git log --oneline menampilkan riwayat Anda. Kode di sebelah kiri seperti a3f91c2 adalah penanda unik commit tersebut.



Menulis pesan commit yang baik

Pesan commit adalah catatan untuk diri Anda sendiri tiga bulan dari sekarang, dan untuk anggota kelompok Anda. Tulis dengan pola: kata kerja, lalu apa yang diubah. Bayangkan Anda melengkapi kalimat "Commit ini akan ...".

Pesan yang baik	Kenapa baik	Pesan yang buruk	Kenapa buruk
Tambah navbar di halaman utama	Jelas apa yang berubah dan di mana	update	Tidak memberi informasi apa pun
Perbaiki tautan menu yang salah	Menyebut masalah yang diperbaiki	fix bug	Bug yang mana? Di berkas mana?

Pisahkan CSS ke berkas terpisah	Menjelaskan alasan perubahan struktur	asdfasdf	Tidak dapat dibaca siapa pun
Hapus kode uji coba yang tertinggal	Jujur menyebut pembersihan	final banget ini	Tidak menjelaskan isi perubahan

Langkah 4 — Menghubungkan ke GitHub dan mengirim (push)

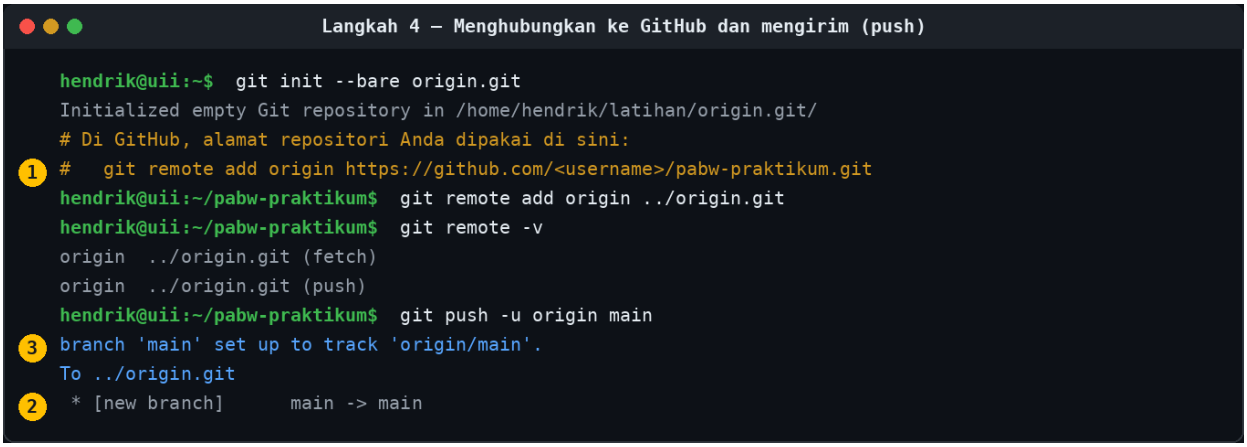
Sampai di sini semuanya masih ada di komputer Anda. Supaya pekerjaan Anda masuk ke GitHub, dua hal harus dilakukan: memberi tahu Git alamat repositori tujuan, lalu mengirim commit ke sana.

Pertama, buat repositori kosong di GitHub dengan nama yang sama (lihat Bagian 2.5). Jangan centang Add a README file kali ini, supaya repositorinya benar-benar kosong dan tidak berbenturan dengan berkas Anda. Setelah dibuat, GitHub menampilkan alamat repositori Anda. Salin alamat itu, lalu jalankan:

```
# Ganti <nama-pengguna> dengan nama pengguna GitHub Anda
git remote add origin https://github.com/<nama-pengguna>/pabw-praktikum.git

git remote -v            # periksa alamat sudah tercatat
git push -u origin main  # kirim commit ke GitHub
```

Penjelasan singkat: origin adalah julukan untuk alamat tersebut, supaya Anda tidak perlu mengetik alamat panjang berulang kali. main adalah nama branch utama. Pilihan -u cukup dipakai sekali di awal; setelah itu Anda dapat menulis git push saja.

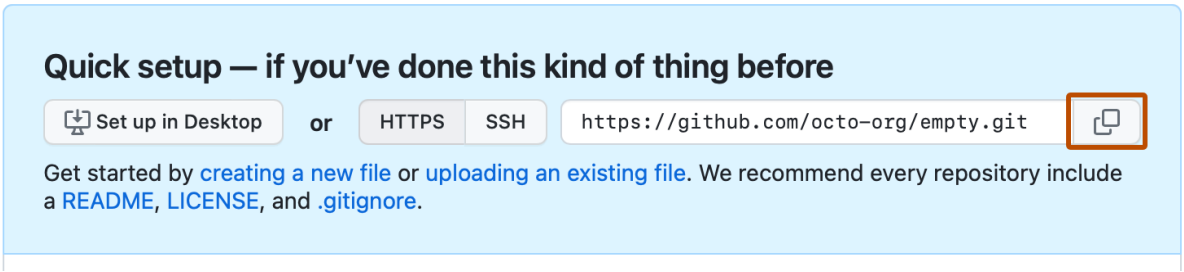


Pada contoh ini alamat tujuannya adalah repositori di komputer yang sama, sehingga terbaca sebagai ../origin.git. Pada tugas Anda, alamatnya berupa tautan GitHub.

Langkah 5 — Menyalin repositori di komputer lain (clone)

Clone dipakai ketika Anda ingin mendapatkan repositori yang sudah ada di GitHub ke komputer baru — misalnya di komputer laboratorium, atau ketika anggota kelompok baru bergabung. Berbeda dengan mengunduh ZIP, clone sekaligus membawa seluruh riwayat, sehingga Anda bisa push dan pull dari sana.

```
# Di GitHub, klik tombol hijau Code lalu salin alamat HTTPS-nya
git clone https://github.com/<nama-pengguna>/pabw-praktikum.git
cd pabw-praktikum
git log --oneline
```



Tombol Code berisi alamat yang dipakai untuk clone. Salin alamat HTTPS untuk memakai cara token, atau pilih tab SSH kalau Anda memakai kunci SSH.

Sumber gambar: GitHub Docs — CC BY 4.0

```
Langkah 5 — Menyalin repositori di komputer lain (clone)

# Di GitHub: git clone https://github.com/<username>/pabw-praktikum.git
hendrik@uii:~$  git clone origin.git salinan
1 Cloning into 'salinan'...
done.
hendrik@uii:~/salinan$  git log --oneline
2 6551482 Tambah halaman index
hendrik@uii:~/salinan$  ls
index.html
```

Langkah 6 — Mengabaikan berkas yang tidak perlu (.gitignore)

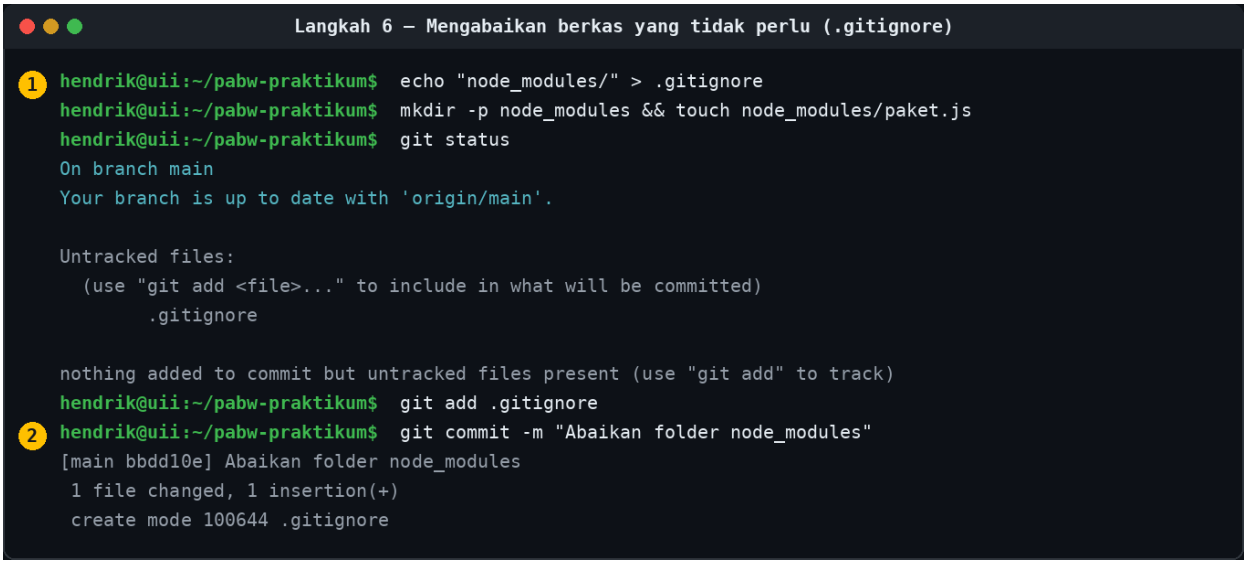
Sebagian berkas tidak boleh masuk repositori: kata sandi, berkas konfigurasi pribadi, dan folder besar seperti node_modules yang isinya lebih dari seribu berkas pustaka. Kalau ikut ter-commit, repositori Anda akan membengkak dan rahasia Anda bisa bocor. Solusinya berkas bernama .gitignore.

```
echo "node_modules/" > .gitignore      # abaikan seluruh folder node_modules

# Pola yang paling sering dipakai, tulis satu per baris:
#   node_modules/      folder pustaka, akan dibuat ulang dengan npm install
#   .env               berkas berisi kata sandi dan kunci rahasia
#   *.log              berkas catatan yang terus bertambah
#   .DS_Store          berkas bawaan macOS
#   dist/  build/      hasil pembangunan otomatis

git add .gitignore
git commit -m "Abaikan folder node_modules"
```

Perhatikan keluaran keduanya: karena node_modules sudah masuk daftar abaikan, Git tidak lagi menyebut folder itu sebagai Untracked files. Berkas .gitignore sendiri tetap harus di-commit agar pengaturan ini ikut terbawa ke komputer anggota kelompok yang lain.



Langkah 7 — Bekerja di cabang sendiri (branch)

Bayangkan dua orang menyunting halaman yang sama pada waktu bersamaan. Kalau keduanya bekerja langsung di branch main, perubahan salah satu orang pasti tertimpa. Karena itu setiap pekerjaan baru dilakukan di cabang sendiri, baru digabungkan setelah selesai dan diperiksa.

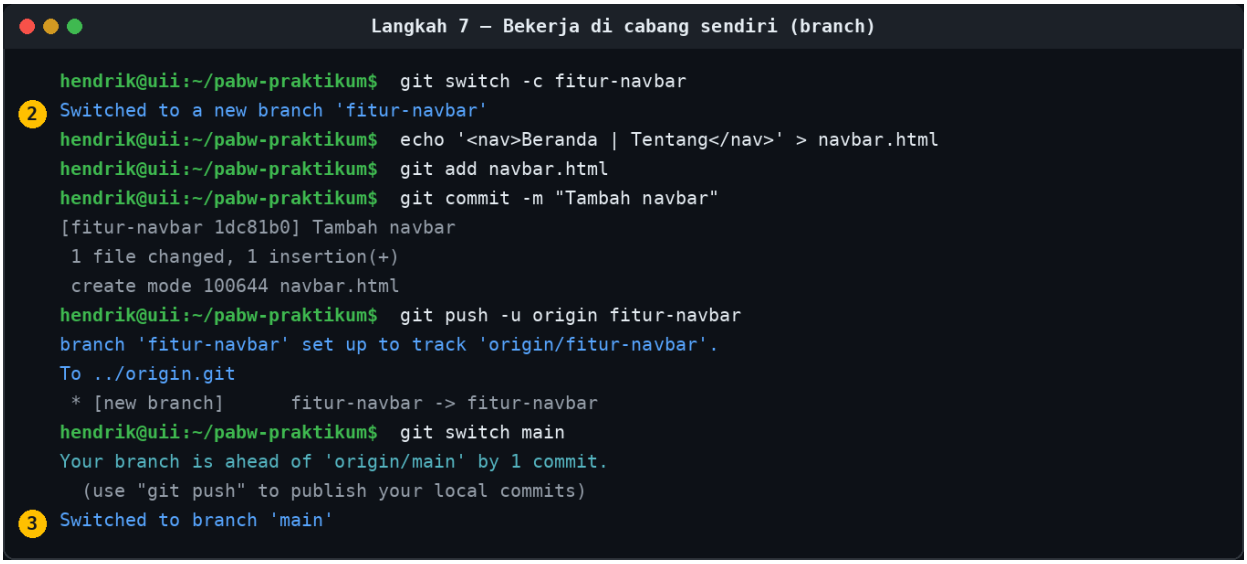
```
git switch -c fitur-navbar      # buat cabang baru sekaligus pindah ke sana

echo "<nav>Beranda | Tentang</nav>" > navbar.html
git add navbar.html
git commit -m "Tambah navbar"

git push -u origin fitur-navbar  # kirim cabang ini ke GitHub
git switch main                  # kembali ke cabang utama
```

Perintah	Kegunaan
git branch	Menampilkan daftar cabang. Tanda bintang menandai cabang tempat Anda berada.
git switch -c nama-cabang	Membuat cabang baru lalu langsung berpindah ke sana.
git switch nama-cabang	Berpindah ke cabang yang sudah ada.
git switch main	Kembali ke cabang utama.
git branch -d nama-cabang	Menghapus cabang yang sudah selesai digabungkan.

Pemberian nama cabang. Pakai huruf kecil dan tanda hubung, dengan pola jenis-pekerjaan dan topiknya. Contoh yang baik: fitur-navbar, perbaikan-tautan-menu, dokumen-readme. Hindari nama seperti coba1, asdf, atau nama Anda sendiri.

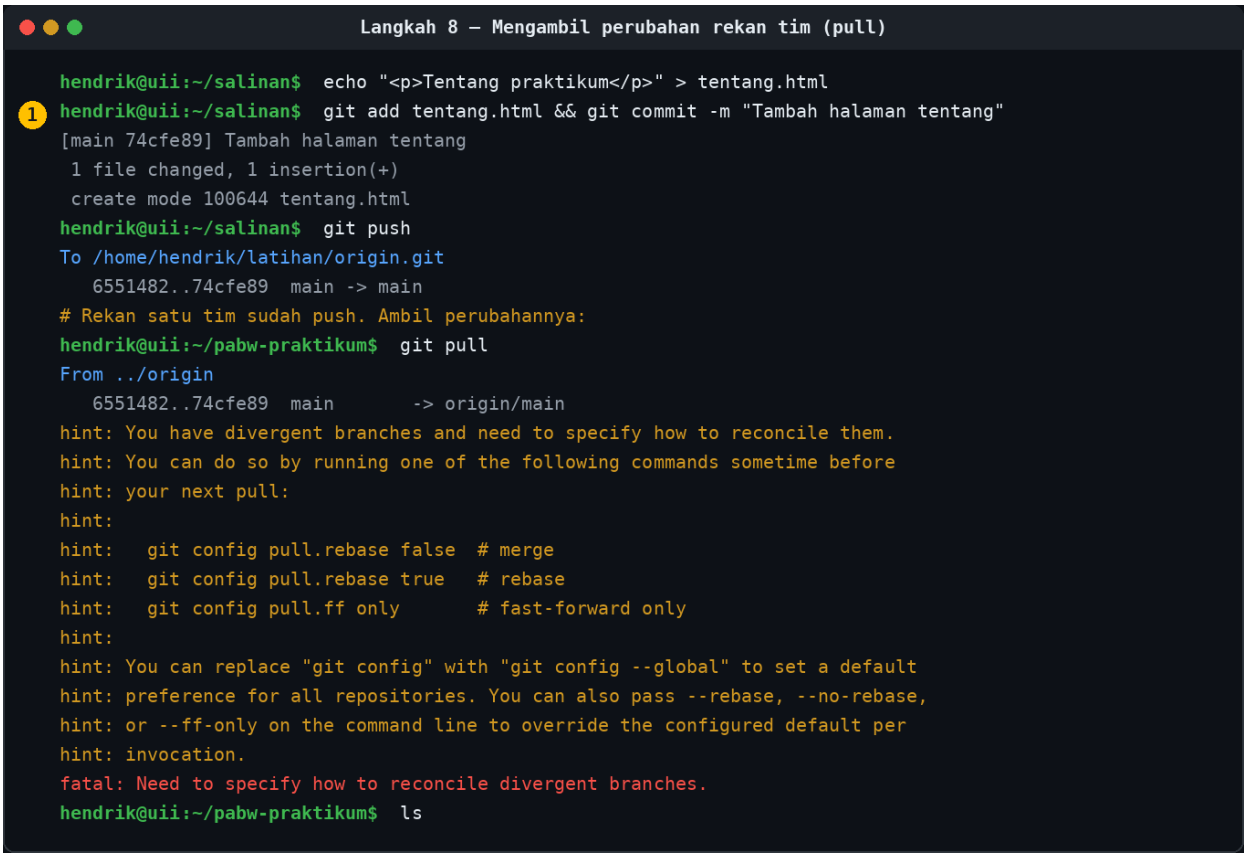


Langkah 8 — Mengambil perubahan rekan tim (pull)

Kebalikan dari push adalah pull. Setiap kali Anda mulai bekerja, ambil dulu perubahan terbaru supaya Anda tidak bekerja di atas versi yang sudah usang.

```
git pull              # ambil dan gabungkan perubahan terbaru dari GitHub
ls                    # periksa berkas baru yang masuk
```

Pada contoh di bawah, rekan Anda telah menambahkan berkas tentang.html lalu mengirimnya. Setelah git pull, berkas itu muncul di komputer Anda. Kata Fast-forward berarti Git cukup memajukan riwayat Anda tanpa perlu penggabungan yang rumit.



```
Langkah 8 — Mengambil perubahan rekan tim (pull)  (lanjutan)

    index.html
    node_modules
```

Biasakan urutan ini setiap kali mulai bekerja. git pull terlebih dahulu, baru mulai mengedit. Kebiasaan ini mencegah sebagian besar konflik sebelum terjadi.

Langkah 9 — Menyelesaikan konflik

Konflik terjadi ketika dua orang mengubah bagian yang sama pada berkas yang sama. Ini bukan kerusakan dan bukan kesalahan Anda; konflik adalah hal yang wajar dan harus diselesaikan. Git sengaja berhenti dan menyerahkan keputusan kepada manusia, karena hanya Andalah yang tahu versi mana yang benar.

Pada contoh berikut, dua cabang mengubah berkas footer.html. Git menolak melanjutkan dan menandai berkas itu sebagai konflik.

```
Langkah 9 — Menyelesaikan konflik

hendrik@uii:~/pabw-praktikum$  git switch -c fitur-footer
Switched to a new branch 'fitur-footer'
hendrik@uii:~/pabw-praktikum$  echo '<footer>Versi A</footer>' > footer.html
hendrik@uii:~/pabw-praktikum$  git add footer.html && git commit -m "Footer versi A"
[fitur-footer 13b5ee1] Footer versi A
 1 file changed, 1 insertion(+)
 create mode 100644 footer.html
hendrik@uii:~/pabw-praktikum$  git switch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)
Switched to branch 'main'
hendrik@uii:~/pabw-praktikum$  echo '<footer>Versi B</footer>' > footer.html
hendrik@uii:~/pabw-praktikum$  git add footer.html && git commit -m "Footer versi B"
[main f38892d] Footer versi B
 1 file changed, 1 insertion(+)
 create mode 100644 footer.html
hendrik@uii:~/pabw-praktikum$  git merge fitur-footer
Auto-merging footer.html
1 CONFLICT (add/add): Merge conflict in footer.html
Automatic merge failed; fix conflicts and then commit the result.
hendrik@uii:~/pabw-praktikum$  git status
On branch main
Your branch and 'origin/main' have diverged,
and have 2 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)
```

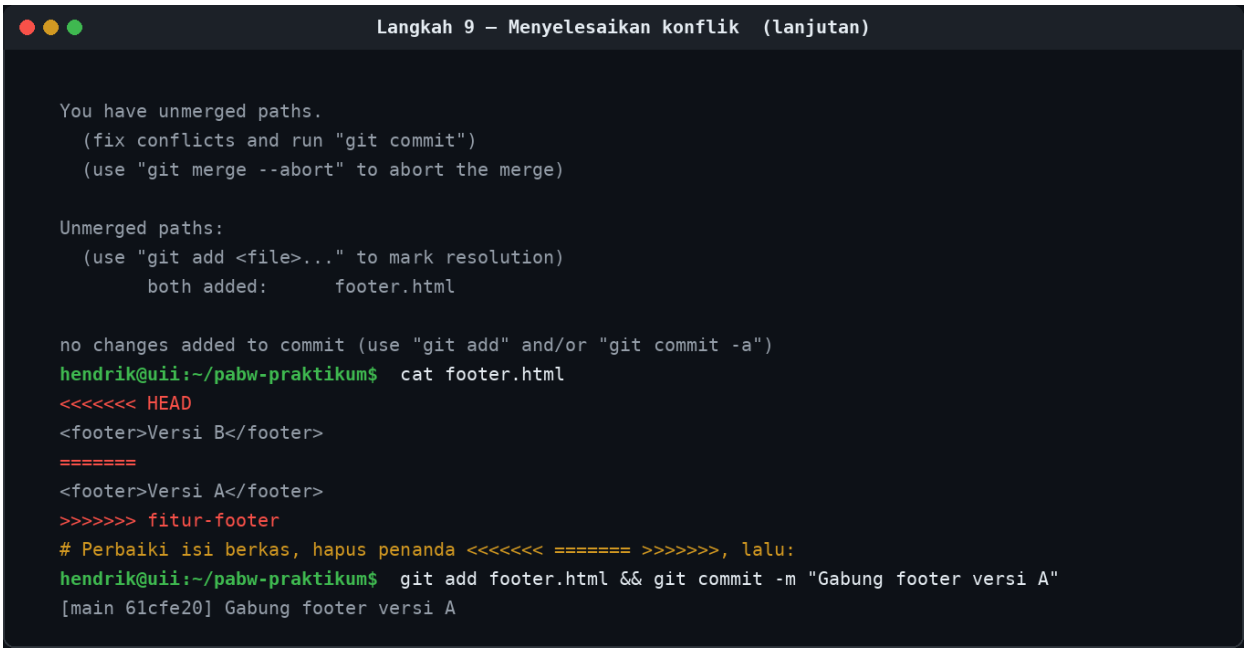
Buka berkas yang bertanda konflik dengan editor teks. Anda akan menemukan penanda seperti ini. Tugas Anda hanya tiga: pilih atau gabungkan isi yang benar, hapus semua baris penanda, lalu simpan berkasnya.

```
<<<<<<< HEAD
<footer>Versi B</footer>
=======
<footer>Versi A</footer>
>>>>>>> fitur-footer

# Cara membacanya:
#   Bagian antara <<<<<<< HEAD dan =======      adalah versi Anda (yang sedang aktif).
#   Bagian antara ======= dan >>>>>>> nama-cabang  adalah versi dari cabang lain.
#   Hapus KETIGA baris penanda itu, sisakan isi yang Anda putuskan.
#
# Hasil akhir setelah diperbaiki, misalnya:
#   <footer>Versi A dan B digabung</footer>
```

Setelah berkas disimpan, tandai bahwa konflik sudah selesai dengan git add, lalu akhiri dengan commit. Perintah git status akan menuntun Anda; Git selalu menampilkan langkah berikutnya yang harus dilakukan.

```
git add footer.html
git status                  # pastikan tidak ada lagi berkas berkonflik
git commit -m "Gabung footer versi A"
```



Kalau keadaan terasa kacau dan Anda ingin membatalkan penggabungan. Jalankan git merge --abort. Git akan mengembalikan keadaan persis seperti sebelum penggabungan dimulai. Tidak ada pekerjaan yang hilang, karena commit Anda tetap tersimpan di cabangnya masing-masing.

BAGIAN 4

Alur kerja kelompok

Bagian ini adalah cara kerja yang akan kita pakai sepanjang semester untuk proyek kelompok. Tujuannya sederhana: tidak ada anggota yang pekerjaannya tertimpa, dan riwayat proyek selalu bisa ditelusuri.

4.1 Aturan yang disepakati

1. **main bersih.** Branch main hanya menerima hasil yang sudah selesai dan diperiksa. Jangan bekerja langsung di sana.
2. **Satu orang satu cabang.** Satu orang, satu branch, satu tugas. Ukuran branch yang kecil membuat konflik jarang terjadi.
3. **Pull sebelum kerja.** Ambil perubahan terbaru sebelum mulai bekerja, supaya Anda tidak bekerja di atas versi lama.
4. **Lewat pull request.** Setiap penggabungan ke main melewati pull request agar ada yang memeriksa.
5. **Commit sering.** Tulis commit kecil-kecil yang jelas, bukan satu commit raksasa di akhir hari.

4.2 Alur harian, langkah demi langkah

Langkah	Yang dilakukan	Perintah
1	Ambil perubahan terbaru dari GitHub	git switch main git pull
2	Buat cabang baru untuk tugas Anda hari ini	git switch -c fitur-nama-tugas
3	Kerjakan, lalu simpan dalam beberapa commit kecil	git add . git commit -m "pesan jelas"
4	Kirim cabang Anda ke GitHub	git push -u origin fitur-nama-tugas
5	Buka pull request di GitHub, minta rekan meninjau	(dilakukan di situs GitHub)
6	Setelah disetujui, gabungkan lalu hapus cabangnya	(dilakukan di situs GitHub)

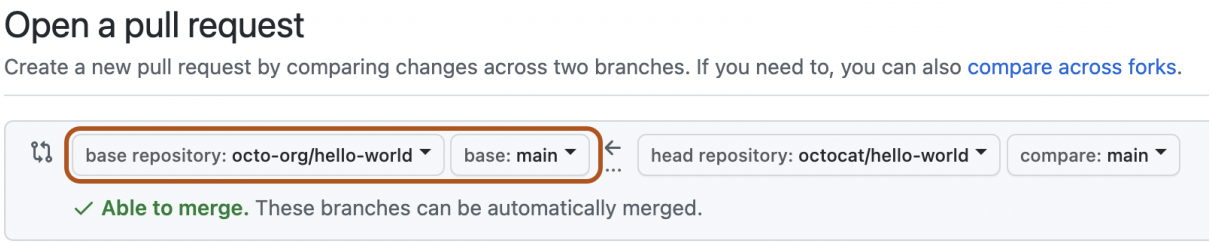
Keterangan tambahan untuk langkah 3: perintah git add . dengan titik berarti memasukkan semua perubahan pada folder tempat Anda berada. Perintah ini praktis, tetapi biasakan memeriksa dengan git status terlebih dahulu supaya berkas yang tidak seharusnya ikut tidak terbawa — terutama berkas berisi kata sandi.

4.3 Membuat pull request di GitHub

Pull request (disingkat PR) adalah cara meminta agar cabang Anda digabungkan ke main, sekaligus tempat anggota lain melihat persis baris mana yang berubah sebelum menyetujuinya.

1. **Buka repositori Anda di GitHub.** — Setelah Anda push sebuah cabang, GitHub biasanya sudah menampilkan tombol Compare & pull request di bagian atas halaman.
2. **Klik Compare & pull request.** — Kalau tombolnya tidak muncul, klik tab Pull requests lalu tombol New pull request.
3. **Pastikan arah penggabungannya benar.** — base: main berarti tujuan, dan compare: fitur-nama-tugas berarti sumber. Anda menggabungkan cabang Anda menuju main.
4. **Tulis judul dan keterangan.** — Judul singkat berisi apa yang dikerjakan. Keterangan berisi apa yang diubah, apa yang sudah dicoba, dan hal yang belum selesai.
5. **Klik Create pull request.** — Jangan klik Merge pull request sebelum ada yang meninjau.
6. **Minta rekan meninjau melalui tab Files changed.** — Mereka dapat memberi komentar pada baris tertentu.

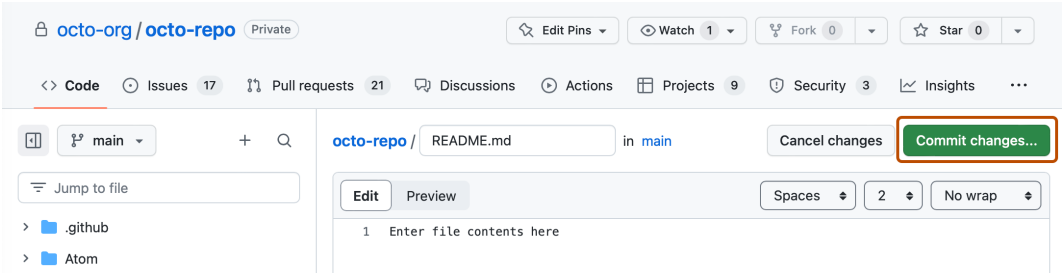
7. **Setelah disetujui, klik Merge pull request, lalu Delete branch.** — Menghapus cabang setelah digabungkan menjaga daftar cabang tetap rapi.



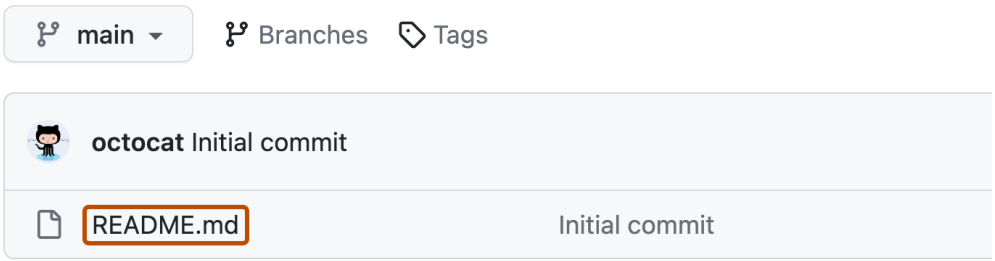
Halaman pembuatan pull request. Bagian base adalah tujuan, compare adalah sumber perubahan yang akan digabungkan.
Sumber gambar: GitHub Docs — CC BY 4.0

4.4 Membuat berkas langsung di situs GitHub

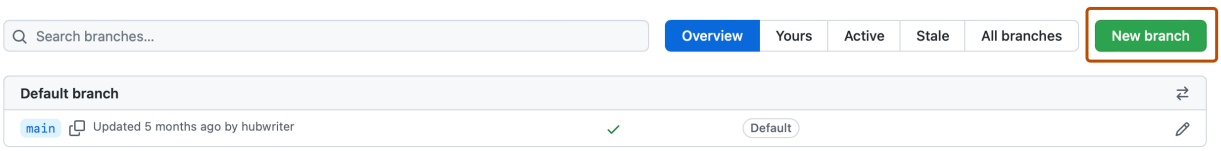
Kadang Anda hanya perlu memperbaiki salah tulis kecil, misalnya pada berkas README, dan tidak ingin membuka terminal. GitHub memungkinkan penyuntingan langsung di situs. Perlu diingat bahwa perubahan ini tetap berupa commit, jadi tetap perlu pesan yang jelas.



Tombol untuk menambah berkas baru atau mengunggah berkas yang sudah ada.
Sumber gambar: GitHub Docs — CC BY 4.0



Berkas README ditampilkan di halaman utama repositori. README adalah halaman depan proyek Anda — tulis penjelasan singkat tentang isi proyek dan anggota kelompok.
Sumber gambar: GitHub Docs — CC BY 4.0



Tombol pemilih cabang di halaman repositori. Dari sini Anda dapat berpindah cabang.
Sumber gambar: GitHub Docs — CC BY 4.0

4.5 Membiasakan diri membaca perbedaan (diff)

Sebelum commit, biasakan melihat apa yang sebenarnya berubah. Baris berawalan tanda kurang berarti dihapus, dan baris berawalan tanda tambah berarti ditambahkan.

```
git diff                # perubahan yang belum di-add
git diff --staged       # perubahan yang sudah di-add, siap di-commit
git log --oneline --graph # riwayat dalam bentuk grafik
git show a3f91c2        # isi satu commit tertentu
```

BAGIAN 5

Mengumpulkan tugas lewat GitHub Classroom

Mata kuliah ini memakai GitHub Classroom untuk tugas praktik dan proyek kelompok. Alurnya sedikit berbeda dari repositori latihan Anda, karena repositori tugas dibuat oleh dosen, bukan oleh Anda.

5.1 Menerima tugas

1. **Klik tautan tugas yang dibagikan dosen atau asisten.** — Tautannya berbentuk invitation link dari classroom.github.com.
2. **Masuk dengan akun GitHub Anda.** — Pastikan memakai akun dengan email yang sama seperti pada pengaturan Git Anda.
3. **Pilih nama Anda dari daftar kelas, atau isi nama pengguna GitHub Anda.** — Satu nama hanya boleh memakai satu akun. Kalau nama Anda sudah berpasangan dengan akun yang salah, hubungi dosen.
4. **Klik Accept this assignment.** — GitHub akan membuatkan repositori pribadi khusus untuk Anda di dalam organisasi kelas. Proses ini butuh beberapa detik.
5. **Buka repositori yang muncul, lalu salin alamatnya.** — Repositori itu hanya bisa Anda dan dosen lihat.
6. **Clone ke komputer Anda.** — git clone <alamat-yang-disalin> — perintahnya sama seperti Langkah 5.

Simpan tautan repositori tugas Anda. Repositori tugas jarang muncul di halaman profil Anda. Simpan tautannya, atau buka classroom.github.com pada daftar kelas untuk menemukannya kembali.

5.2 Mengerjakan dan mengumpulkan

Setelah repositori tugas berada di komputer Anda, cara bekerjanya sama seperti latihan: edit berkas, git add, git commit, git push. Yang perlu diperhatikan hanyalah tiga hal berikut, karena berkaitan dengan penilaian.

Yang diperhatikan	Penjelasan	Akibat kalau diabaikan
Commit yang jelas dan bertahap	Dosen dapat melihat proses kerja Anda, bukan hanya hasil akhir	Nilai proses tidak dapat diberikan; terlihat seperti hasil salin-tempel
Push, bukan hanya commit	Commit hanya tersimpan di komputer Anda	Tugas dianggap tidak dikumpulkan, walaupun Anda sudah bekerja berjam-jam
Pengungkapan penggunaan AI	Penggunaan AI wajib diungkapkan pada README atau laporan	Dianggap pelanggaran integritas akademik
Jangan mengubah berkas .github/workflows	Di dalamnya ada sistem penilaian otomatis milik dosen	Penilaian otomatis gagal berjalan

Memastikan unggahan Anda benar-benar berhasil

Jangan mengandalkan perasaan "sepertinya sudah terkirim". Lakukan pemeriksaan berikut setiap kali selesai mengumpulkan.

```
git status                # pastikan tidak ada perubahan yang belum di-commit
git log origin/main -1    # commit terakhir yang SUDAH ada di GitHub
git push                  # bila ada yang tertinggal, kirim sekarang

# Lalu buka halaman repositori Anda di GitHub dan periksa:
#   1. Berkas yang Anda kerjakan sudah tampil di sana.
#   2. Pesan commit terakhir adalah yang Anda buat barusan.
#   3. Waktu commit sesuai dengan waktu Anda mengerjakan.
```

Kalau ketiga hal itu terpenuhi, tugas Anda benar-benar sudah masuk. Kalau berkas tidak terlihat di GitHub, hampir selalu penyebabnya adalah git push yang belum dijalankan.

BAGIAN 6

Kesalahan umum dan cara memperbaikinya

Daftar ini disusun dari kekeliruan yang paling sering terjadi pada pemula. Cari pesan kesalahan Anda pada kolom pertama, lalu jalankan perbaikan pada kolom terakhir.

6.1 Saat melakukan commit

Pesan atau gejala	Artinya	Cara memperbaiki
fatal: not a git repository	Anda menjalankan perintah Git di folder yang belum dipasangi Git, atau berada di folder yang salah	Periksa posisi Anda dengan pwd, lalu masuk ke folder proyek dengan cd. Kalau memang belum ada, jalankan git init.
nothing to commit, working tree clean	Git tidak menemukan perubahan apa pun. Biasanya berkas belum disimpan di editor, atau sudah di-commit sebelumnya	Simpan berkas di editor (Ctrl+S), lalu periksa dengan git status.
Please tell me who you are	Identitas Git belum diatur, sehingga Git menolak membuat commit	Jalankan git config --global user.name "Nama" dan user.email, lalu ulangi commit.
Berkas yang seharusnya diabaikan ikut ter-commit, misalnya .env	Berkas itu sudah tercatat Git sebelum masuk .gitignore, sehingga daftar abaikan tidak berpengaruh	Hentikan pelacakan dengan git rm --cached nama-berkas, pastikan namanya sudah ada di .gitignore, lalu commit ulang.

6.2 Saat mengirim dan mengambil (push, pull)

Pesan atau gejala	Artinya	Cara memperbaiki
Authentication failed	Kata sandi atau token salah, atau Anda memakai kata sandi akun padahal GitHub sudah tidak mengizinkannya	Buat Personal Access Token (Bagian 2.6) dan pakai token itu sebagai kata sandi.
Support for password authentication was removed	GitHub sudah tidak menerima kata sandi akun untuk push	Ganti ke Personal Access Token atau kunci SSH.
Permission denied (publickey)	Kunci SSH belum terpasang atau belum ditempelkan ke GitHub	Jalankan ssh -T git@github.com untuk menguji, lalu tempelkan isi ~/.ssh/id_ed25519.pub ke GitHub.
Updates were rejected because the remote contains work that you do not have locally	Rekan Anda sudah mengirim commit lebih dahulu, sehingga versi Anda tertinggal	Jalankan git pull lebih dahulu, selesaikan konflik bila ada, baru git push kembali. Jangan pakai force push.
failed to push some refs	Alamat repositori tujuan belum benar, atau nama branch tidak cocok	Periksa dengan git remote -v, lalu gunakan git push -u origin main untuk pertama kali.
Everything up-to-date, tetapi berkas tidak muncul di GitHub	Perintah push dijalankan dari folder yang salah, atau commit belum dibuat	Jalankan git status dan git log untuk memastikan Anda berada di repositori yang benar dan commit sudah ada.

6.3 Saat bekerja dengan cabang dan konflik

Pesan atau gejala	Artinya	Cara memperbaiki
CONFLICT (content): Merge conflict in nama-berkas	Dua orang mengubah bagian yang sama pada satu berkas. Ini wajar, bukan kerusakan	Buka berkasnya, pilih isi yang benar, hapus baris penanda <<<<<<< ======= >>>>>>>, lalu git add dan git commit.

		Lihat Langkah 9.
error: Your local changes would be overwritten by merge	Anda punya perubahan yang belum di-commit, sehingga Git menolak menggabungkan	Commit atau simpan perubahan itu dahulu. Untuk membuangnya: git restore nama-berkas.
You are in detached HEAD state	Anda berada di luar cabang mana pun, biasanya karena berpindah ke commit tertentu	Kembali ke jalur aman dengan git switch main.
git switch gagal karena perubahan belum disimpan	Ada penyuntingan aktif yang akan hilang saat berpindah cabang	Commit dulu, atau hentikan sementara dengan git stash, kemudian lanjutkan dengan git stash pop.
Cabang sudah digabungkan tetapi masih tampil di daftar	Cabang tidak dihapus otomatis setelah merge	Hapus dengan git branch -d nama-cabang setelah pull terakhir.

6.4 Hati-hati: perintah yang bisa menghapus pekerjaan

Baca dulu sebelum menjalankan. Empat perintah berikut ini benar-benar menghilangkan pekerjaan yang belum disimpan. Tidak ada tombol pembatalan. Jangan menjalankannya kalau Anda belum yakin, dan jangan pernah menjalankannya pada repositori tugas orang lain.

Perintah	Akibatnya	Kapan masih boleh dipakai
git reset --hard	Membuang semua perubahan yang belum di-commit, tanpa bisa dikembalikan	Hanya bila Anda benar-benar yakin ingin membuang pekerjaan yang belum tersimpan
git push --force	Menimpa riwayat di GitHub, sehingga commit orang lain bisa hilang	Hampir tidak pernah. Pada proyek kerja sama, ini dilarang. Kalau perlu, gunakan --force-with-lease dan katakan dulu kepada kelompok
git clean -fd	Menghapus semua berkas yang belum dilacak Git	Hanya bila Anda tahu pasti berkas apa saja yang akan terhapus. Coba dulu dengan git clean -nd yang hanya menampilkan daftar
git checkout . atau git restore .	Membuang semua perubahan yang belum di-commit di seluruh proyek	Hanya bila perubahan itu memang tidak diperlukan

BAGIAN 7

Kartu ringkasan perintah

Halaman ini boleh dicetak dan ditempel di dekat meja belajar. Perintah yang dipakai sehari-hari hanya sebelas.

7.1 Yang dipakai setiap hari

Perintah	Artinya
git status	Sedang ada perubahan apa? Tanyakan ini setiap kali bingung.
git add nama-berkas	Pilih berkas yang akan disimpan ke dalam commit berikutnya.
git add .	Pilih semua perubahan di folder ini.
git commit -m "pesan"	Simpan pilihan itu sebagai satu titik riwayat.
git pull	Ambil perubahan terbaru dari GitHub. Jalankan sebelum mulai bekerja.
git push	Kirim commit Anda ke GitHub.
git log --oneline	Lihat daftar riwayat pekerjaan.
git switch -c nama-cabang	Buat cabang baru untuk pekerjaan baru.
git switch main	Kembali ke cabang utama.
git diff	Lihat persis baris mana yang berubah.
git restore nama-berkas	Batalkan perubahan yang belum di-commit pada berkas itu.

7.2 Untuk keperluan tertentu

Perintah	Artinya
git config --global --list	Periksa identitas dan pengaturan Git Anda.
git remote -v	Periksa alamat repositori GitHub yang terhubung.
git branch	Lihat daftar cabang dan cabang yang sedang aktif.
git clone <alamat>	Salin repositori dari GitHub ke komputer ini.
git stash	Simpan sementara pekerjaan yang belum selesai, agar bisa berpindah cabang.
git stash pop	Ambil kembali pekerjaan yang disimpan sementara itu.
git merge --abort	Batalkan penggabungan yang sedang berkonflik.
git rm --cached nama-berkas	Berhenti melacak berkas, tetapi berkasnya tetap ada di komputer.

7.3 Urutan lengkap pertama kali memakai repositori baru

```
git config --global user.name  "Nama Anda"        # sekali seumur pemasangan
git config --global user.email "email@uii.ac.id"   # sekali seumur pemasangan

git clone <alamat-repositori>                  # ambil repo ke komputer
cd nama-repositori                             # masuk ke foldernya
git switch -c fitur-tugas-pertama              # buat cabang sendiri

git add .                                      # pilih semua perubahan
git commit -m "Tambah halaman profil"          # simpan sebagai riwayat
git push -u origin fitur-tugas-pertama         # kirim ke GitHub

# Selanjutnya buka GitHub dan buat pull request.
```

BAGIAN A

Lampiran A — Daftar periksa sebelum mengumpulkan tugas

Jalankan daftar ini setiap kali sebelum menutup laptop. Menghemat banyak masalah.

- ☐ Saya sudah menjalankan git status dan tidak ada perubahan yang tertinggal belum di-commit.
- ☐ Saya sudah menjalankan git push, bukan hanya git commit.
- ☐ Saya sudah membuka halaman repositori di GitHub dan melihat berkas terbaru saya di sana.
- ☐ Riwayat commit saya terdiri atas beberapa langkah kecil dengan pesan yang jelas, bukan satu commit besar di akhir.
- ☐ Tidak ada berkas berisi kata sandi, token, atau kunci rahasia yang ikut ter-push (periksa berkas .env).
- ☐ Saya sudah menuliskan pengungkapan penggunaan AI pada README bila memakainya.
- ☐ Saya tidak mengubah berkas di dalam folder .github.
- ☐ Saat bekerja berkelompok, pekerjaan saya berada di cabang sendiri dan sudah diajukan lewat pull request.

BAGIAN B

Lampiran B — Glosarium

Istilah	Penjelasan
Branch	Cabang pengembangan. Jalur terpisah dari main, dipakai agar pekerjaan yang belum selesai tidak mengganggu versi utama.
Clone	Menyalin repositori beserta seluruh riwayatnya dari GitHub ke komputer, satu kali di awal.
Commit	Satu titik dalam riwayat, berisi gambaran seluruh proyek pada satu waktu beserta pesan penjelasan.
Conflict	Keadaan ketika dua perubahan pada bagian yang sama tidak dapat digabungkan otomatis, sehingga perlu diputuskan manusia.
Diff	Tampilan perbedaan antar versi. Baris bertanda kurang dihapus, baris bertanda tambah ditambahkan.
Fast-forward	Penggabungan paling sederhana: Git cukup memajukan riwayat tanpa perlu menyatukan dua jalur.
Fork	Salinan repositori milik orang lain ke akun Anda sendiri. Tidak dipakai dalam mata kuliah ini.
Git	Program pencatat riwayat perubahan yang berjalan di komputer Anda.
GitHub	Layanan web tempat salinan repositori ditaruh agar dapat dibagikan dan dikerjakan bersama.
GitHub Classroom	Fitur GitHub untuk kelas: dosen membagikan tugas, dan setiap mahasiswa memperoleh repositori sendiri.
.gitignore	Berkas berisi daftar pola nama berkas dan folder yang tidak boleh dilacak Git.
HEAD	Penunjuk posisi Anda saat ini. Biasanya menunjuk pada commit terakhir di cabang yang sedang aktif.
main	Nama branch utama, tempat versi resmi proyek berada.
Merge	Menggabungkan perubahan dari satu cabang ke cabang lain.
Origin	Julukan bawaan untuk alamat repositori GitHub yang terhubung dengan repositori lokal Anda.
Pull	Mengambil perubahan terbaru dari GitHub dan menggabungkannya ke cabang Anda.
Pull request	Permintaan agar cabang Anda digabungkan ke main, sekaligus tempat peninjauan oleh anggota lain.
Push	Mengirim commit dari komputer Anda ke GitHub.
Remote	Repositori yang tersimpan di server, yaitu GitHub, sebagai pasangan dari repositori lokal Anda.
Repositori	Folder proyek yang di dalamnya Git sudah dipasang, berisi berkas dan seluruh riwayatnya.
Staging area	Ruang tunggu: perubahan yang sudah dipilih dengan git add dan akan masuk ke commit berikutnya.
Working directory	Folder kerja tempat Anda menyunting berkas, sebelum dicatat Git.

BAGIAN C

Lampiran C — Sumber gambar

Gambar keluaran terminal dalam panduan ini adalah tangkapan layar sungguhan dari perintah Git yang dijalankan pada contoh proyek. Alamat repositori pada contoh tersebut menunjuk ke repositori latihan di komputer yang sama, sehingga tidak memuat kredensial GitHub siapa pun. Pada tugas Anda, alamatnya berupa tautan GitHub Anda sendiri.

Tangkapan layar antarmuka GitHub diambil dari materi resmi GitHub Docs yang diterbitkan di repositori github/docs dengan lisensi Creative Commons Attribution 4.0 International (CC BY 4.0). Hak cipta tangkapan layar tersebut tetap milik GitHub, Inc. Sumber aslinya dapat diakses melalui https://github.com/github/docs pada folder assets/images.

Materi acuan mata kuliah ini adalah Rencana Pembelajaran Semester PABW 2026/2027 Program Studi Informatika, Universitas Islam Indonesia.

Panduan ini disusun untuk keperluan pengajaran PABW 2026/2027.